

Skriftlig prøve: 29 maj 2024

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Skriftlige hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–25%, Opgave 2–25%, Opgave 3–25%, Opgave 4–25%

Note: Hvis du mangler specifikationer, kan du tilføje dine egne og fortsætte opgaveløsningen. Ikke alle begreber/beskrivelser er oversat til dansk. Eksempelvis er ordene "runtime stack" ikke oversat. I enkelte tilfælde er ikke oversatte betegnelser anført i anførselstegn ("...").

Opgave 1 (25%)

Besvar de følgende spørgsmål:

- a) Konverter følgende LC3-assemblerprogram til LC3-maskinkode:

```
.ORIG x3000
    ADD R7, R2, #12
LOOP  BRnzp LOOP
    .END
```

Binary

0001 111 010 101100

0000 111 1111111111

To hex

0001 1110 1010 1100 -> x1EAC

0000 1111 1111 1111 -> x0FFF

- b) Hvad er den hexadecimale værdi svarende til binærstrengen "1100000011011110"?

1100 0000 1101 1110 -> xCODE

- c) Repræsenter det decimale tal -23 som en 8-bit binær streng, kodet ved hjælp af 2's komplement repræsentation.

+23 = 00010111 -> -23=11101001

- d) Hvilken opcode er ikke gyldig for LC3-instruktionssættet brugt i lærebogen? Giv et eksempel på en 16-bit binær streng, der ikke svarer til en gyldig LC3-instruktion.

Ugyldig opkode: 1101

Ugyldig instruktion: 1101101001100100

Opgave 2 (25%)

Givet nedenstående LC3 assemblerkode:

```
.ORIG x3000
    AND R0, R0, #0 ; R0 = t
    ADD R1, R0, #15 ; R1 = x
aa    BRnz dd
    AND R2, R1, #7
    BRnp cc
    AND R3, R3, #0 ; R3 = z
bb    ADD R0, R0, R3
```

Skriftlig prøve: 29 maj 2024

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Skriftlige hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–25%, Opgave 2–25%, Opgave 3–25%, Opgave 4–25%

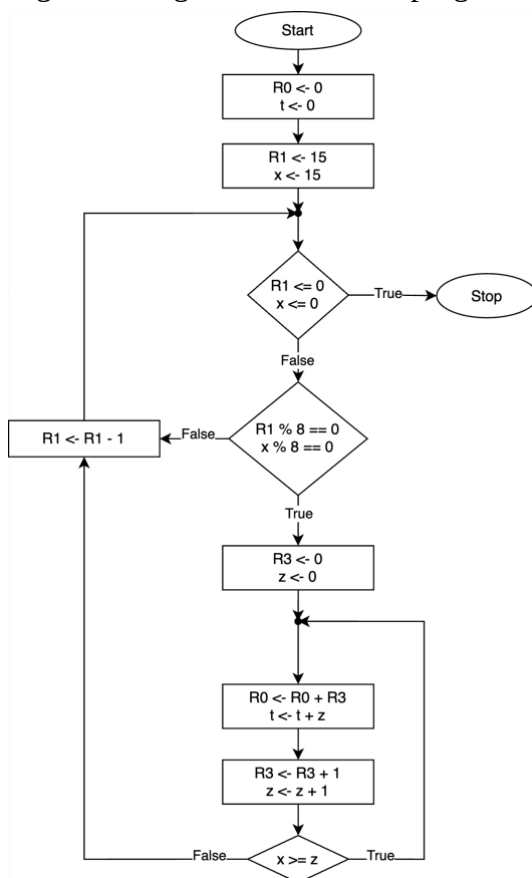
```

        ADD R3, R3, #1
        NOT R4, R3
        ADD R4, R4, #1
        ADD R4, R1, R4
        BRzp bb
cc      ADD R1, R1, #-1
        BRnzp aa
dd      HALT
.END
    
```

a) Hvad er symboltabellen for dette program?

aa	x3002
bb	x3006
cc	x300C
dd	X300E

b) Tegn flowdiagrammet for dette program.



Skriftlig prøve: 29 maj 2024

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Skriftlige hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–25%, Opgave 2–25%, Opgave 3–25%, Opgave 4–25%

- c) Konverter dette LC3-assemblerprogram til et tilsvarende C-program. Når du skriver C-programmet, skal du allokere 3 heltalsvariabler t, x og z, hvor t svarer til værdien gemt i R0, x svarer til værdien gemt i R1, og z svarer til værdien gemt i R3.

```
#include <stdio.h>

void main() {
    int x=15;
    int t=0, z;
    while(x>0) {
        if (x%8==0) {
            z=0;
            do{
                t=t+z;
                z++;
            } while (x>=z);
        }
        x--;
    }
}
```

Opgave 3 (25%)

Givet nedenstående C program:

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to print the linked list
void printList(struct Node* node) {
    while (node != NULL) {
        printf("%d ", node->data);
        node = node->next;
    }
}
```

Skriftlig prøve: 29 maj 2024

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Skriftlige hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–25%, Opgave 2–25%, Opgave 3–25%, Opgave 4–25%

```
    printf("\n");
}

// Function to push a new node at the beginning
void push(struct Node** head_ref, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = new_data;
    new_node->next = (*head_ref);
    (*head_ref) = new_node;
}

// Function to find the middle of the linked list
// You need to fill in the missing code in this function
struct Node* findMiddle(struct Node* head) {
    // Fill in the missing code here
}

int main() {
    struct Node* head = NULL;

    // Creating a linked list 17->2->9->6->4
    push(&head, 4);
    push(&head, 6);
    push(&head, 9);
    push(&head, 2);
    push(&head, 17);

    printf("Linked List: \n");
    printList(head);

    struct Node* middle = findMiddle(head);

    if (middle != NULL) {
        printf("Middle element is: %d\n", middle->data);
    } else {
        printf("The linked list is empty.\n");
    }

    return 0;
}
```

Du får en "linked list", der er initialiseret i funktionen **main**. Din opgave er at skrive en funktion, der finder elementet på den midterste position i "linked list"en. Hvis "linked list"en har

Skriftlig prøve: 29 maj 2024

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Skriftlige hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–25%, Opgave 2–25%, Opgave 3–25%, Opgave 4–25%

et lige antal elementer, skal du returnere det andet af de to midterste elementer. Du skal udfylde den manglende kode for at fuldføre funktionen **findMiddle**.

For eksempel:

- Hvis "linked list"en indeholder værdierne 17->2->9->6->4, skal funktionen **findMiddle** returnere en pointer til noden, der indeholder værdien 9.
 - Hvis "linked list"en indeholder værdierne 17->2->9->6->4->1, skal funktionen **findMiddle** returnere en pointer til noden, der indeholder værdien 6.
- a) Udfyld den manglende kode i **findMiddle**-funktionen for korrekt at finde midterelementet (ud fra position) i "linked list"en.

```
struct Node* findMiddle(struct Node* head) {  
    struct Node* slow = head;  
    struct Node* fast = head;  
  
    while (fast != NULL && fast->next != NULL) {  
        slow = slow->next;  
        fast = fast->next->next;  
    }  
  
    return slow;  
}
```

Opgave 4 (25%)

Følgende C program ønskes kørt på en **LC3 processor**:

```
#include <stdio.h>  
  
struct S{  
    char c;  
    int* v;  
};  
  
int main() {  
    int arr[]={22, 33, 11, 25};  
    struct S s1;  
    struct S* ps;  
    struct S** pps;  
    int d=7;  
  
    pps=&ps;
```

Skriftlig prøve: 29 maj 2024

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Skriftlige hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–25%, Opgave 2–25%, Opgave 3–25%, Opgave 4–25%

```

ps=&s1;
(*pps)->v=arr+2;
*(ps->v)++;
s1.v = s1.v + 1;
*(ps->v)++;
(*ps).c='D';
(*pps)->v=&d;
*((*pps)->v)=5;
(*pps)->c='a';
d++;

printf("arr = {%d, %d, %d, %d}\n", arr[0],arr[1],arr[2],arr[3]);
printf("s1.c = %c\n", s1.c);
printf("*s1.v = %d\n", *(s1.v));

return 0;
}

```

- a) Hvad er den maksimale anvendte størrelse (i '16-bit words') for "runtime stack", når vi kører programmet?

12 (3 bookkeeping, 2 pointers, 2 for the struct s1, 4 for the array, 1 for d)

- b) Vis indholdet af "activation record" for funktionen **main** lige før den sidste linje i koden (return 0;) bliver kørt. Det antages, at "stack pointer"en i R6 har værdien FE00, når "runtime stack"en er tom.

xFDF4	d	7 -> 5->6
xFDF5	pps	? -> xFDF6
xFDF6	ps	? -> xFDF7
xFDF7	s1.c	? -> 'D' -> 'a'
xFDF8	s1.v	? -> xFDFB->FDFF->FDF4
xFDF9	arr[0]	22
xFDFA	arr[1]	33
xFDFB	arr[2]	11 -> 12
xFDFC	arr[3]	25 -> 26
xFDFD	DL	
xFDFE	RA	
xFDFE	RV	
xFE00		

Skriftlig prøve: 29 maj 2024

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpe midler: Skriftlige hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–25%, Opgave 2–25%, Opgave 3–25%, Opgave 4–25%

- c) Hvad er outputtet af dette program? Med andre ord, hvad bliver skrevet i terminalvinduet, når dette program køres?

```
arr = {22, 33, 12, 26}  
s1.c= a  
*(s1.v)= 6
```

Hjælp til besvarelsen:

For et C-program som kører på en LC3 processor gælder:

- Integer og pointer variabler er 16 bit (1 word)
- Hukommelsen er 16-bit (1 word) adresserbar
- Lokale variable placeres på “runtime stack”en i samme rækkefølge som de erklæres
- Rækkefølgen, hvormed argumenterne til et funktionskald placeres på stakken, går fra højre mod venstre

Kursus nummer: 02322

Vægtning: Opgave 1–25%, Opgave 2–25%, Opgave 3–25%, Opgave 4–25%

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD ⁺	0001				DR			SR1			0	00		SR2		
ADD ⁺	0001				DR			SR1			1	imm5				
AND ⁺	0101				DR			SR1			0	00		SR2		
AND ⁺	0101				DR			SR1			1	imm5				
BR	0000				n	z	p	PCoffset9								
JMP	1100				000			BaseR			000000					
JSR	0100				1	PCoffset11										
JSRR	0100				0	00		BaseR			000000					
LD ⁺	0010				DR			PCoffset9								
LDI ⁺	1010				DR			i			PCoffset9					
LDR ⁺	0110				DR			BaseR			offset6					
LEA	1110				DR			PCoffset9								
NOT ⁺	1001				DR			SR			111111					
RET	1100				000			111			000000					
RTI	1000				00000000000000											
ST	0011				SR			PCoffset9								
STI	1011				SR			PCoffset9								
STR	0111				SR			BaseR			offset6					
TRAP	1111				0000			trapvect8								